SASKEN TECHNOLOGIES

Student Technical Revision Notes

Medium-level, topic-wise notes across all eight preparation sections

Section	Coverage
C Programming	Language fundamentals, pointers, memory, strings, structures, preprocessor, bitwise operations
C++ Programming	OOP, object lifetime, polymorphism, STL, templates, exceptions, modern C++
Operating Systems	Processes, threads, scheduling, synchronization, deadlocks, memory management
DBMS	Relational model, keys, SQL, normalization, transactions, indexing
Computer Networks	OSI/TCP-IP, addressing, protocols, TCP/UDP, DNS, DHCP, TLS
Embedded Systems	MCUs, firmware, interrupts, timers, memory-mapped I/O, DMA, RTOS basics
Software Engineering	SDLC, testing, debugging, version control, build tools, quality
DSA Concepts	Complexity, arrays, strings, linked lists, stacks, queues, trees, graphs, hashing, algorithms

How to use these notes: Read the concept summaries, memorize the key distinctions and formulas, then practise the examples and MCQs from the separate question bank. These notes are intended for revision—not as a replacement for hands-on coding.

Recruitment context: The shared Sasken email specifically names C or C++ programming, technical topics, abstract reasoning and English communication, followed by technical and HR interviews. The exact 2027 test blueprint has not been supplied. The remaining subjects are included as useful technical-interview preparation.

Prepared as a student-facing revision companion • Medium-level coverage

How to revise efficiently

Use a repeatable cycle: understand → recall without notes → solve → explain. For each topic, students should be able to define it, compare it with a related concept, give one example and identify a common mistake.

Suggested 8-section revision sequence

Order	Section	What to be able to do
1	C programming	Trace expressions, reason about pointers and write safe functions.
2	C++ programming	Explain OOP and object lifetime; use STL appropriately.
3	Operating Systems	Explain process/thread behaviour, synchronization and virtual memory.
4	DBMS	Write basic SQL and explain keys, normalization and ACID.
5	Computer Networks	Explain layers and trace a basic client-server connection.
6	Embedded Systems	Explain MCU peripherals, interrupts and real-time constraints.
7	Software Engineering	Explain testing levels, debugging and version control.
8	DSA concepts	Choose data structures and estimate time/space complexity.

Core exam habits

- For code questions, trace values in a table rather than guessing the output.
- Check operator precedence, data types, integer division, pointer validity and array bounds.
- For coding problems, state the approach first, then complexity, then edge cases.
- For MCQs, eliminate options that violate a language rule or confuse two different concepts.
- In interviews, explain assumptions and trade-offs. If uncertain, reason aloud rather than bluffing.

Quick complexity reminder

Complexity	Typical interpretation	Example
O(1)	Constant work	Array index access
O(log n)	Problem size repeatedly shrinks	Binary search
O(n)	One pass through input	Linear search
O(n log n)	Divide-and-conquer sorting	Merge sort, average quicksort
$O(n^2)$	Nested full scans	Simple bubble sort
O(2■)	Subsets / branching recursion	Naive subset enumeration

1. C Programming

C is a procedural, statically typed language widely used where low-level control, predictable performance and hardware access matter. Sasken's shared role description explicitly asks for programming in C or C++ and DSA implementation.

1.1 Program structure, types and expressions

- A C program is translated through preprocessing, compilation, assembly and linking. A translation unit is a source file after preprocessing, including included headers.
- main is the entry point in a hosted C program. Include standard headers for declarations, e.g. stdio.h for printf and stdlib.h for malloc/free.
- Fundamental types include char, integer types, floating types and _Bool. Exact sizes are implementation-dependent; use sizeof rather than assuming int is 4 bytes.
- Integer promotions and usual arithmetic conversions can change the type used in an expression. Be careful when mixing signed and unsigned values.
- Integer division truncates toward zero: 7/2 is 3 and -7/2 is -3 when both operands are int.
- Operator precedence decides grouping; it does not define evaluation order. Use parentheses and avoid multiple side effects in one expression.

1.2 Operators and bit manipulation

Operator	Meaning	Example
&	Bitwise AND	5 & 3 = 1 (0101 & 0011 = 0001)
|	Bitwise OR	5 | 3 = 7
^	Bitwise XOR	5 ^ 3 = 6
~	Bitwise complement	Flips every bit of the promoted operand
<<	Left shift	Unsigned x << 1 shifts bits left
>>	Right shift	Unsigned x >> 1 shifts right with zero fill

Common bit tricks (use unsigned values)

Check bit k: (x & (1u << k)) != 0
Set bit k: x |= (1u << k)
Clear bit k: x &= ~(1u << k)
Toggle bit k: x ^= (1u << k)
Check odd: (x & 1u) != 0

Caution: shifting by a negative count or by a count at least the width of the promoted left operand is undefined. Signed overflow is undefined; unsigned arithmetic wraps modulo 2■.

C Programming – Pointers, arrays and memory

1.3 Arrays, strings and pointers

- An array stores a fixed number of same-type elements contiguously. Valid indices are 0 through length–1.
- In most expressions, an array expression converts to a pointer to its first element. An array is not itself a pointer; sizeof(array) in its defining scope returns the whole array size.
- In a function parameter, int a[] is adjusted to int *a. Pass the length separately because the function does not receive the full array size.
- A C string is a char array terminated by '\0'. The storage must include space for the terminator.
- strlen counts characters before '\0'; sizeof measures object storage size. For char s[] = "cat", sizeof(s) is 4 while strlen(s) is 3.
- A pointer stores an address. &x; obtains an address; *p accesses the pointed-to object. Pointer arithmetic advances by units of the pointed-to type.
- Never dereference NULL, an uninitialized pointer, a dangling pointer or a pointer outside the permitted object range.

1.4 Pointer distinctions

Term	Meaning / interview point
NULL pointer	A pointer value that does not point to an object/function; compare p == NULL.
Void pointer	Generic object pointer; convert to a suitable object-pointer type before dereferencing.
Wild pointer	Uninitialized pointer with an indeterminate value.
Dangling pointer	Pointer whose target object's lifetime has ended or storage has been released.
Double pointer	Pointer to a pointer, e.g. int **pp; useful when a function must update a caller's pointer.
const int *p	Cannot modify the pointed-to int through p; p can be reassigned.
int * const p	p cannot be reassigned; pointed-to int can be modified.

1.5 Dynamic memory management

Function	Purpose	Important rule
malloc(n)	Allocates n bytes	Contents uninitialized; check for NULL.
calloc(count,size)	Allocates array storage	All bytes initialized to zero.
realloc(p,n)	Resizes allocation	Use a temporary pointer; on failure original block remains valid.
free(p)	Releases allocated storage	free(NULL) is safe; do not use freed storage.

Safe realloc pattern

```
tmp = realloc(ptr, new_size);
if (tmp != NULL) { ptr = tmp; }
else { /* ptr still refers to the original allocation */ }
```

Memory errors to recognize: leak (not released), use-after-free, double free, out-of-bounds access and returning the address of a local automatic variable.

C Programming – functions, aggregates and pitfalls

1.6 Functions, scope and storage duration

- A function prototype declares the return type and parameter types before a call. C passes arguments by value; passing a pointer allows a function to modify the pointed-to object.
- Recursion requires a base case and progress toward it. Deep recursion can exhaust the call stack.
- A block-scope automatic variable normally exists during its block execution. A static local variable retains its value for the entire program execution.
- A file-scope static function or object has internal linkage (visible within that translation unit). extern commonly declares an entity defined elsewhere.
- const restricts modification through a particular access path; volatile concerns observable accesses and is not a thread-synchronization mechanism.

1.7 Structures, unions, enums and preprocessor

- A struct gives each member its own storage, subject to padding and alignment. A union's members share storage; its size is sufficient for its largest member plus alignment requirements.
- enum defines named integer constants. typedef creates a type alias; it does not create a distinct new type.
- #include inserts header contents during preprocessing. Header guards prevent repeated inclusion within a translation unit.
- #define creates macro substitutions. # stringifies a macro argument and ## pastes preprocessing tokens.
- Macros can cause precedence and side-effect bugs. Prefer inline functions or typed constructs when practical.

1.8 High-value C pitfalls

Pitfall	What to remember
Signed overflow	Undefined behaviour; do not rely on wraparound.
Unsigned overflow	Defined modulo 2■ for the unsigned type.
Unsequenced side effects	May cause undefined behaviour, e.g. modifying the same scalar more than once without sequencing.
String literal write	Modifying a string literal has undefined behaviour.
scanf("%s", ...)	Can overflow; use a width limit or fgets.
strcpy / strcat	Caller must ensure destination has enough space.
sizeof parameter	An array parameter is a pointer, so sizeof gives pointer size there.

C interview checklist

- Explain pointer arithmetic, array decay, const pointer vs pointer-to-const.
- Trace malloc/calloc/realloc/free and identify leaks or dangling pointers.
- Explain stack vs heap, struct vs union, static vs extern and volatile vs const.
- Write safe string, array and bit-manipulation functions.

2. C++ Programming

C++ supports procedural, object-oriented and generic programming. It builds on C-like syntax but has classes, references, templates, exceptions, the Standard Library and resource-management abstractions.

2.1 Classes, objects and access control

- A class defines a type; an object is an instance of that type. A class groups state (data members) and behaviour (member functions).
- class members are private by default; struct members are public by default. Access levels are public, protected and private.
- Encapsulation controls access to internal state. Abstraction exposes essential operations while hiding implementation details.
- A constructor initializes an object; a destructor releases resources as the object's lifetime ends. Constructors cannot be virtual.
- Use member-initializer lists, especially for const members, reference members and base classes.

2.2 The four OOP pillars

Concept	Meaning	Example
Encapsulation	Bundle data and operations; control access	Private balance with deposit/withdraw methods
Abstraction	Expose what an object does, hide how	A device API hides register-level details
Inheritance	Derive a new type from an existing type	ElectricCar derives from Vehicle
Polymorphism	Same interface, different behaviour	Virtual draw() implemented by Circle and Rectangle

2.3 Overloading, overriding and dispatch

- Overloading: same function name with different parameter lists; generally resolved at compile time.
- Overriding: derived class provides a matching implementation of a virtual base function.
- A virtual function enables runtime dispatch through a base pointer/reference. Use override so the compiler checks that an override really occurs.
- A pure virtual function is declared with = 0 and makes a class abstract. Abstract classes can still have constructors.
- A polymorphic base class usually needs a virtual destructor if objects may be deleted through base pointers.
- During base construction/destruction, virtual calls do not dispatch to more-derived portions that are not yet constructed or have already been destroyed.

Example

```
struct Base { virtual void show() { } virtual ~Base() = default; };
struct Derived : Base { void show() override { } };
Base* p = new Derived();
p->show(); // calls Derived::show()
delete p; // safe because Base destructor is virtual
```

C++ Programming – object lifetime and modern C++

2.4 Pointers, references and casts

Feature	Pointer	Reference
Can be null?	Yes	A valid reference is expected to refer to an object/function
Can be reseated?	Yes	No; remains bound to its referent
Syntax	int *p	int &r;
Access	Dereference with *	Use like the referred object

- new allocates storage and constructs an object; delete destroys and releases a single object. Use new[]/delete[] in matching pairs for arrays.
- Prefer standard containers and smart pointers over manual new/delete in ordinary application code.
- static_cast performs specified compile-time conversions; dynamic_cast performs checked polymorphic downcasts; const_cast changes cv-qualification; reinterpret_cast is low-level and does not guarantee safe access.

2.5 RAII and smart pointers

- RAII ties resource ownership to object lifetime: acquire in construction, release in destruction. It makes cleanup reliable during normal return and exception unwinding.
- unique_ptr represents exclusive ownership and is move-only. shared_ptr represents shared ownership using a control block/reference count. weak_ptr observes shared ownership without extending lifetime.
- shared_ptr cycles can keep objects alive forever; use weak_ptr for non-owning links in ownership graphs.
- Rule of Three: if a class needs a destructor, copy constructor or copy assignment for resource management, it may need all three. Rule of Five adds move constructor and move assignment. Rule of Zero prefers member types that manage resources themselves.

2.6 STL containers – selection guide

Container	Use case / typical complexity
vector	Contiguous dynamic array; random access O(1), append amortized O(1), middle insert/erase O(n).
list	Doubly linked list; insert/erase at known iterator O(1), random access O(n).
deque	Double-ended sequence; efficient push/pop at both ends, random access.
map / set	Ordered tree containers; search/insert/erase typically O(log n).
unordered_map / unordered_set	Hash-based; average O(1), worst-case O(n).
stack / queue	Container adaptors for LIFO / FIFO behaviour.

Iterator invalidation

vector reallocation invalidates pointers, references and iterators to its elements. Insertion/erasure can invalidate positions at or after the modified point. Always check the container's invalidation rules.

C++ Programming – templates, exceptions and concurrency

2.7 Templates and generic programming

- Function templates and class templates allow code to work with different types while retaining compile-time type checking.
- Template argument deduction infers template parameters from function arguments. Variadic templates use parameter packs to accept variable numbers of arguments.
- SFINAE means Substitution Failure Is Not An Error: in certain template contexts, an invalid substitution removes a candidate from overload resolution.
- Concepts (C++20) express constraints on template parameters and can make template diagnostics clearer.
- STL algorithms operate on iterator ranges, e.g. sort(v.begin(), v.end()) for random-access iterators.

2.8 Exceptions and error handling

- Use try to enclose code that may throw, throw to signal an exception and catch to handle a matching exception.
- Catch exceptions by const reference in many polymorphic exception cases to avoid slicing and unnecessary copies.
- Destructors should generally not allow exceptions to escape. Stack objects are destroyed during stack unwinding.
- noexcept declares a non-throwing contract; an exception escaping a noexcept function calls std::terminate.

2.9 Move semantics and value categories

- An lvalue identifies an object with identity; an rvalue is a value category commonly associated with temporaries. C++ value categories are more detailed than this shorthand.
- T&& is an rvalue reference. In template deduction, T&& can become a forwarding reference.
- std::move is a cast that enables move overload selection; it does not itself move data.
- std::forward preserves the value category of a forwarding-reference argument.
- Move operations can transfer resources rather than deep-copy them, improving performance for resource-owning types.

2.10 Concurrency basics

- A data race occurs when conflicting unsynchronized accesses from different threads happen and at least one is a write; in C++ this is undefined behaviour.
- std::mutex protects critical sections. std::lock_guard and std::unique_lock use RAII to manage locks.
- std::atomic provides atomic operations on supported types but does not automatically make a multi-step algorithm atomic.
- Avoid holding locks longer than necessary; define a consistent lock order to reduce deadlock risk.

C++ interview checklist

- Compare overloading vs overriding, class vs struct, pointer vs reference and map vs unordered_map.
- Explain virtual destructors, object slicing, RAII, Rule of Five and smart-pointer ownership.
- Explain why vector iterators can be invalidated and when to choose list, map or unordered_map.
- Know what move, forward, templates, exceptions, mutex and atomic are for.

3. Operating Systems

An operating system manages hardware resources and provides abstractions such as processes, threads, files, virtual memory and system calls.

3.1 Processes and threads

Process	Thread
Program in execution with its own virtual address space and resources	Execution path within a process; threads share process code/data/resources
Higher cost to create/context-switch in many systems	Usually lighter-weight than a process
Inter-process communication may be needed to share data	Shared memory is commonly available within the same process
Has process-level resources and at least one thread	Has its own execution state, registers and stack

- A process commonly moves among new, ready, running, waiting/blocked and terminated states.
- A context switch saves the current task state and restores another task's state. It has overhead and does not perform useful application work itself.
- A system call is a controlled interface for requesting kernel services such as file I/O, process creation and memory mapping.

3.2 CPU scheduling

Algorithm	Core idea	Strength / concern
FCFS	First arrival runs first	Simple; may cause convoy effect.
SJF	Shortest CPU burst first	Low average waiting time if burst lengths known; long jobs may starve.
SRTF	Preemptive SJF	New shorter remaining job can preempt; more context switches.
Priority	Highest priority selected	Starvation possible; aging can help.
Round Robin	Each process gets a time quantum	Fair responsiveness; too-small quantum increases overhead.

Scheduling formulas

Turnaround time = Completion time – Arrival time
Waiting time = Turnaround time – CPU burst time
Response time = First CPU start time – Arrival time

Example: a job arrives at time 2, first runs at 5 and completes at 14 after using 6 units of CPU. Response time = 5–2 = 3; turnaround = 14–2 = 12; waiting = 12–6 = 6.

Operating Systems – synchronization and deadlocks

3.3 Concurrency and synchronization

- A race condition occurs when the result depends on the timing/interleaving of concurrent operations.
- A critical section accesses shared state and must be protected so concurrent execution does not corrupt data.
- A mutex provides mutual exclusion: one thread owns the lock at a time. A semaphore is a counter-based synchronization primitive used for signalling or managing a pool of resources.
- A binary semaphore can have values 0/1, while a counting semaphore can represent multiple available instances. A mutex normally has ownership semantics.
- Busy waiting repeatedly checks a condition and can waste CPU. Blocking synchronization lets a thread sleep until it can proceed.

3.4 Deadlocks

A deadlock occurs when a set of processes/threads are permanently blocked because each is waiting for a resource held by another.

Necessary condition	Meaning
Mutual exclusion	At least one resource is non-shareable.
Hold and wait	A task holds one resource while requesting another.
No preemption	Resources cannot be forcibly taken away under the model.
Circular wait	A cycle exists in the resource-wait graph.

Deadlock handling approaches

- Prevention: design the system so at least one necessary condition cannot hold.
- Avoidance: grant requests only if the system remains in a safe state (e.g. Banker's algorithm model).
- Detection and recovery: detect cycles or deadlocked sets, then abort or preempt as appropriate.
- Practical lock-ordering rules and timeouts can reduce risk in software.

3.5 Memory management

- Virtual memory gives each process an address-space abstraction. The MMU and page tables translate virtual addresses to physical frames.
- Paging divides virtual memory into fixed-size pages and physical memory into frames. A page table records mappings and protection information.
- A page fault occurs when an access requires OS handling because the mapping/page is not present or violates a protection rule. Not every page fault is an error.
- A TLB (Translation Lookaside Buffer) caches recent address translations to reduce page-table lookup overhead.
- Internal fragmentation is wasted space inside an allocated block; external fragmentation is free space split into separate holes.

Operating Systems – memory, files and quick review

3.6 Stack, heap and virtual memory

Region / concept	Typical role
Stack	Call frames, return addresses and many automatic local variables; per-thread in common systems.
Heap	Dynamic allocations whose lifetime is controlled by allocator APIs or language runtime.
Text/code	Executable instructions; often mapped read-only/executable.
Data/BSS	Initialized and zero-initialized static-storage objects.
Virtual address space	Per-process address abstraction; may map code, libraries, heap, stack and memory-mapped regions.

3.7 File systems and I/O

- A file system organizes persistent data into files and directories and manages metadata, permissions and storage allocation.
- Buffering and caching reduce I/O overhead. DMA can transfer data between a device and memory with limited CPU copying.
- Interrupt-driven I/O lets a device notify the CPU when service is needed; polling repeatedly checks device status.

3.8 Frequently confused terms

Term	Remember
Concurrency	Multiple tasks make progress over overlapping periods; not necessarily simultaneous.
Parallelism	Tasks execute at the same instant on multiple processing resources.
Starvation	A task waits indefinitely because others are repeatedly favoured.
Aging	Gradually raises waiting tasks' priority to reduce starvation.
Thrashing	System spends excessive time paging rather than executing useful work.
Race condition	Correctness depends on timing/interleaving.
Deadlock	Tasks wait permanently on one another's resources.

OS interview checklist

- Compare process and thread; explain context switching and system calls.
- Calculate waiting, turnaround and response times from a scheduling table.
- Explain mutex vs semaphore, race condition, critical section and the four deadlock conditions.
- Explain paging, page faults, TLB, virtual memory, stack vs heap and fragmentation.

4. Database Management Systems

A DBMS stores, organizes, retrieves and protects data. Relational databases represent data in tables and use SQL to define, query and modify it.

4.1 Relational model and keys

- A relation is represented as a table; a row is a tuple and a column is an attribute. A domain defines allowed values for an attribute.
- A superkey uniquely identifies a row. A candidate key is a minimal superkey. One candidate key is selected as the primary key.
- A primary key must be unique and NOT NULL. A foreign key references a candidate/unique key and enforces referential integrity.
- A composite key uses more than one attribute. A surrogate key is an artificial identifier such as an auto-generated ID.
- Entity integrity concerns primary-key values; referential integrity concerns valid foreign-key references.

4.2 SQL command families

Family	Commands / purpose
DDL	CREATE, ALTER, DROP – define or change schema objects.
DML	INSERT, UPDATE, DELETE – change table data.
DQL	SELECT – retrieve data (often grouped under DML in some classifications).
DCL	GRANT, REVOKE – permissions.
TCL	COMMIT, ROLLBACK, SAVEPOINT – transaction control.

4.3 SELECT query order and clauses

```
SELECT columns
FROM table(s)
WHERE row_condition
GROUP BY grouping_columns
HAVING group_condition
ORDER BY sort_columns;
```

- WHERE filters rows before grouping; HAVING filters groups after GROUP BY.
- COUNT(*) counts rows; COUNT(column) ignores NULL values in that column. SUM/AVG generally ignore NULL values.
- INNER JOIN returns matching rows. LEFT JOIN preserves all left rows and fills unmatched right columns with NULL.
- UNION removes duplicates; UNION ALL preserves duplicates. Both require compatible column counts and types.
- NULL means missing/unknown/not applicable, not zero or an empty string. Use IS NULL / IS NOT NULL, not = NULL.

DBMS – normalization, transactions and indexing

4.4 Normalization

Form	Main idea
1NF	Attributes contain atomic values; no repeating groups in the relational design.
2NF	In 1NF and no non-prime attribute depends on a proper subset of a candidate key (no partial dependency).
3NF	In 2NF and no inappropriate transitive dependency of non-key attributes on keys; formal definition uses functional dependencies.
BCNF	For every non-trivial functional dependency X → Y, X is a superkey.

Why normalize?

- Reduce duplicate storage and update anomalies: insertion anomaly, update anomaly and deletion anomaly.
- Normalization decomposes tables while aiming to preserve information and dependencies. Denormalization may be used deliberately for read performance.

4.5 Transactions and ACID

Property	Meaning
Atomicity	All operations in a transaction happen, or none do.
Consistency	A committed transaction preserves database constraints/invariants.
Isolation	Concurrent transactions are controlled so their effects meet the selected isolation level.
Durability	Committed changes survive failures according to the system's guarantees.

Isolation phenomena

- Dirty read: read uncommitted data that may later be rolled back.
- Non-repeatable read: rereading a row returns a different committed value.
- Phantom read: repeating a predicate query returns a changed set of rows.
- Serializable is the strongest standard isolation level; actual behaviour and implementation vary by DBMS.

4.6 Indexes and query performance

- An index is an auxiliary data structure that speeds up searches at the cost of storage and extra write/update work.
- B-tree/B+ tree indexes support ordered lookup and range queries, typically logarithmic height.
- Hash indexes can be effective for equality lookups but generally do not support ordered range scans.
- A composite index on (A, B) is typically most useful for predicates beginning with A; index order matters.
- Use EXPLAIN/plan tools to inspect access paths. Avoid assuming every query should use an index.

DBMS interview checklist

- Explain primary, candidate, super and foreign keys.
- Write SELECT with joins, GROUP BY, HAVING and ORDER BY.
- Explain 1NF, 2NF, 3NF, BCNF and common anomalies.
- Explain ACID, isolation anomalies, indexes and UNION vs UNION ALL.

5. Computer Networks

A computer network connects devices so they can exchange data. Network questions often test layered architecture, addressing, common protocols and what happens when a user opens a website.

5.1 OSI model

Layer	Role	Examples
7 Application	Network services used by applications	HTTP, DNS, SMTP
6 Presentation	Data representation, encoding, encryption concepts	TLS-related presentation functions, formats
5 Session	Session/dialogue management	Session control concepts
4 Transport	End-to-end delivery, ports, reliability/flow control	TCP, UDP
3 Network	Logical addressing and routing	IP, routers, ICMP
2 Data Link	Local-link framing and MAC addressing	Ethernet, Wi-Fi MAC, switches
1 Physical	Signals and bit transmission	Cables, radio, connectors

5.2 TCP/IP model

A common four-layer view: Application (HTTP/DNS), Transport (TCP/UDP), Internet (IP/ICMP) and Link/Network Access (Ethernet/Wi-Fi). Mapping to OSI is approximate; models serve as teaching abstractions.

5.3 Addressing basics

- An IPv4 address is 32 bits; IPv6 uses 128 bits. CIDR notation such as /24 specifies the network prefix length.
- A subnet mask identifies network and host bits. Hosts use a default gateway to send traffic outside the local subnet.
- A MAC address is a link-layer identifier; an IP address is a logical network-layer address. They are not interchangeable.
- A port identifies an application endpoint on a host. A socket commonly combines transport protocol, IP address and port.

Common ports to remember

Service	Common port
HTTP	TCP 80
HTTPS	TCP 443
DNS	UDP/TCP 53
SSH	TCP 22
FTP control	TCP 21
SMTP	TCP 25 (submission commonly 587)
DHCPv4	UDP 67 server / 68 client

Computer Networks – protocols and connection flow

5.4 TCP vs UDP

TCP	UDP
Connection-oriented; handshake establishes connection	Connectionless datagrams
Reliable, ordered byte stream; retransmission and flow/congestion control	No built-in guarantee of delivery, order or retransmission
More protocol overhead	Lower protocol overhead
Common for web, file transfer, many application protocols	Common for real-time media, DNS queries and some games/services

5.5 Important protocols

- DNS resolves domain names to records such as A (IPv4) and AAAA (IPv6). It is a distributed hierarchical naming system.
- DHCP dynamically provides host configuration such as IP address, subnet mask, default gateway and DNS server.
- ARP resolves an IPv4 address to a link-layer address on a local network. IPv6 uses Neighbor Discovery instead of ARP.
- ICMP carries network control and diagnostic messages; ping commonly uses ICMP echo request/reply.
- TLS protects data in transit using encryption and integrity and can authenticate the server with certificates.

5.6 TCP three-way handshake

1. Client → Server: SYN
2. Server → Client: SYN-ACK
3. Client → Server: ACK

This establishes sequence-number state and connection parameters before application data is exchanged.

5.7 What happens when a user opens https://example.com?

- The browser checks relevant caches and resolves the hostname through DNS if needed.
- The client determines the route; local traffic may use ARP/Neighbor Discovery to find the next-hop link address.
- A transport connection is established (commonly TCP, or QUIC over UDP for HTTP/3).
- TLS authenticates the server and negotiates cryptographic keys for a secure session.
- The browser sends an HTTP request; the server responds; the browser processes resources and renders the page.

Networking interview checklist

- Recite OSI layers and give one protocol/device example for each.
- Compare TCP vs UDP, MAC vs IP, DNS vs DHCP and HTTP vs HTTPS.
- Explain subnet prefix, default gateway, ports, sockets and the TCP handshake.
- Describe the high-level flow of opening a secure website.

6. Embedded Systems

An embedded system is a computer-based system designed to perform a dedicated function within a larger product. Sasken's business domains include automotive, industrial, smart devices and communication products, making embedded concepts useful background.

6.1 Microprocessor vs microcontroller

Microprocessor	Microcontroller
Primarily CPU; often uses external memory/peripherals	Typically integrates CPU, RAM, flash and peripherals on one chip
Used in general-purpose/high-performance computing designs	Used in control-oriented and dedicated embedded applications
System design may require more external components	Often lower cost, compact and power-efficient for control tasks

6.2 Embedded system building blocks

- CPU/core executes instructions; flash/ROM stores program code; RAM stores runtime data; EEPROM/flash may store persistent settings.
- GPIO pins provide digital input/output. ADC converts analog signals to digital values; DAC converts digital values to analog signals.
- Timers/counters measure intervals, generate periodic events and support PWM (pulse-width modulation).
- Communication peripherals include UART, SPI, I²C, CAN, USB and Ethernet, selected according to speed, topology, distance and reliability needs.
- Sensors measure physical quantities; actuators produce actions such as movement, switching or display output.

6.3 Common communication interfaces

Interface	Key idea
UART	Asynchronous serial communication using TX/RX; baud rate must match.
SPI	Synchronous serial bus; clock plus data lines and usually chip-select; full-duplex possible.
I²C	Two-wire synchronous bus (SCL/SDA), address-based multi-device communication.
CAN	Robust message-based bus used widely in automotive/industrial systems; arbitration by message identifier.
PWM	Controls average power/voltage using duty cycle of a periodic digital signal.

Embedded Systems – real-time behaviour and hardware interaction

6.4 Interrupts and polling

Interrupt-driven	Polling
Hardware/software event signals CPU to run an interrupt handler	CPU repeatedly checks a status flag or condition
Good responsiveness without constant checking	Simple, but can waste CPU cycles
ISR should be short; defer lengthy work	Polling frequency affects response time and CPU load

Interrupt service routine (ISR) guidelines

- Keep the ISR short and predictable. Capture/clear the event and defer heavy work to a task or main loop.
- Avoid blocking calls, long loops and unsafe shared-state updates inside an ISR.
- Use volatile for memory-mapped registers or values changed outside ordinary program flow when required by the implementation, but remember volatile does not make accesses atomic or thread-safe.
- Protect data shared between ISR and foreground code using suitable atomic or critical-section mechanisms for the target.

6.5 Memory-mapped I/O and DMA

- Memory-mapped I/O exposes peripheral control/status registers at addresses in the processor's address space.
- Register access may require exact-width reads/writes and ordering rules from the hardware documentation.
- DMA (Direct Memory Access) transfers data between peripherals and memory with limited CPU involvement; software must manage buffers, ownership and completion events.

6.6 Real-time operating systems (RTOS)

- A real-time system is judged by correctness of both result and timing. Hard real-time deadlines must not be missed; soft real-time systems tolerate occasional lateness with degraded quality.
- An RTOS provides tasks/threads, scheduling, synchronization, timers and inter-task communication.
- Priority inversion occurs when a high-priority task is blocked by a lower-priority task holding a needed resource; priority inheritance can mitigate it.
- Determinism, bounded latency, worst-case execution time and deadline analysis matter more than average speed alone.

6.7 Embedded interview checklist

- Compare microprocessor and microcontroller; explain firmware and non-volatile memory.
- Compare interrupt vs polling, UART vs SPI vs I²C and hard vs soft real-time.
- Explain ADC, DAC, GPIO, PWM, watchdog timer, DMA and memory-mapped I/O.
- Explain why ISRs should be short and why volatile alone is not synchronization.

7. Software Engineering

Software engineering applies systematic methods to build, test, deploy and maintain reliable software. A product-engineering role can involve development, integration, testing, debugging and sustenance.

7.1 SDLC and development approaches

Approach	Description / trade-off
Waterfall	Sequential phases; works when requirements are stable, but change can be costly.
Agile	Iterative delivery, feedback and adaptation; commonly organized into short cycles.
Scrum	Agile framework using roles/accountabilities, events and product backlog.
V-model	Testing activities are mapped to corresponding development stages.
DevOps	Combines development and operations practices to improve delivery and operational feedback.

7.2 Requirements and design

- Functional requirements describe what the system must do. Non-functional requirements describe qualities such as performance, reliability, security, usability and scalability.
- A good requirement is clear, testable, feasible, traceable and unambiguous.
- High cohesion is desirable: a module should have a focused responsibility. Low coupling is desirable: modules should depend minimally on one another.
- Interfaces define contracts between components. Integration defects often occur where assumptions about data formats, errors, timing or ownership differ.

7.3 Build and toolchain concepts

- Compiler translates source code into another representation, commonly object code. Assembler converts assembly language to machine code.
- Linker combines object files/libraries and resolves external symbols. Loader maps the executable into memory for execution.
- A compile-time error prevents successful translation; a runtime error occurs during execution; a logical error produces incorrect behaviour despite successful execution.
- Warnings identify suspicious code; treat warnings seriously, especially in C/C++ systems programming.

7.4 Version control with Git

```
git status – inspect working tree
git add <file> – stage changes
git commit -m "message" – record a commit
git branch – list/create branches
git switch <branch> – switch branches
git merge <branch> – integrate branch changes
git pull – fetch and integrate remote changes
git push – publish local commits
```

Software Engineering – testing, debugging and quality

7.5 Testing levels and types

Test type	Purpose
Unit testing	Test an individual function/class/component in isolation.
Integration testing	Test interactions between modules or services.
System testing	Test the complete integrated system against requirements.
Acceptance testing	Confirm the product meets user/business acceptance criteria.
Regression testing	Re-run tests after changes to detect unintended breakage.
Smoke testing	Quickly verify that key functions/build are stable enough for further testing.
Performance testing	Measure response time, throughput, resource use and scalability.
Security testing	Look for vulnerabilities, weak controls and unsafe data handling.

7.6 Test design techniques

- Equivalence partitioning divides inputs into groups expected to behave similarly; test representative values.
- Boundary-value analysis focuses on edges: minimum, maximum, just below and just above valid ranges.
- Decision tables test combinations of conditions and resulting actions.
- State-transition testing checks behaviour as a system moves between states.
- For array/string code, test empty input, one element, duplicates, negative values, maximum size and invalid input.

7.7 Debugging workflow

- Reproduce the issue consistently and record input, environment and expected vs actual behaviour.
- Narrow the failure using logs, debugger breakpoints, watch variables and a minimal reproducer.
- Identify the root cause rather than patching only the visible symptom.
- Fix the defect, add a regression test and review nearby code for the same failure pattern.

7.8 Quality attributes

Attribute	Meaning
Reliability	Ability to operate correctly over time.
Maintainability	Ease of understanding, modifying and testing.
Portability	Ease of moving across platforms/environments.
Scalability	Ability to handle increased workload.
Security	Protection of confidentiality, integrity and availability.
Performance	Efficient use of time and resources.

Software engineering interview checklist

- Explain SDLC, Agile, unit vs integration vs system testing and regression testing.
- Compare compiler, assembler, linker and loader.
- Describe Git workflow, merge conflicts and why code reviews matter.
- Explain how you would reproduce, isolate, fix and prevent a software defect.

8. Data Structures & Algorithms

DSA is about choosing a representation and algorithm that solve a problem correctly and efficiently. Students should explain the data structure choice, algorithm steps, time complexity, space complexity and edge cases.

8.1 Complexity analysis

- Time complexity describes how operation count grows with input size n; space complexity describes additional memory growth.
- Big-O is an asymptotic upper bound; Big-Theta is a tight asymptotic bound; Big-Omega is a lower bound.
- Ignore constant factors and lower-order terms for asymptotic comparison: $3n^2 + 4n + 10$ is $O(n^2)$.
- Best, average and worst cases may differ. State which case you are describing.
- Amortized analysis averages cost over a sequence of operations; vector append is amortized O(1), though an individual reallocation can cost O(n).

Complexity	Typical example
O(1)	Array access by index, stack push (array with capacity)
O(log n)	Binary search
O(n)	Linear scan, linked-list traversal
O(n log n)	Merge sort, heap sort
$O(n^2)$	Two nested loops over n elements
O(2■)	Naive subset generation / some recursive search
O(n!)	Generating all permutations

8.2 Arrays and strings

- Arrays provide O(1) indexed access due to contiguous storage. Searching an unsorted array is O(n); inserting/deleting in the middle is O(n) due to shifting.
- Common patterns: two pointers, sliding window, prefix sums, frequency counting and sorting.
- String problems often reduce to character counts, two-pointer scans, sliding windows or hashing.
- For a fixed-size alphabet, a frequency array can provide O(n) time and O(1) auxiliary space relative to input length (alphabet size is constant).

Common array patterns

Pattern	When useful
Two pointers	Sorted pair-sum, reversing, removing duplicates, partitioning.
Sliding window	Contiguous subarray/substring with a moving range.
Prefix sum	Repeated range-sum queries; subarray sum reasoning.
Hash map/set	Fast membership, frequency counts, complement lookup.
Binary search	Monotonic sorted/partitioned search space.

DSA – linked lists, stacks, queues and hashing

8.3 Linked lists

- A singly linked-list node stores data and a pointer to the next node. A doubly linked list also stores a previous pointer.
- Access/search is O(n); insertion at the head is O(1); insertion after a known node is O(1).
- Common interview problems: reverse a list, find middle node, detect a cycle, merge sorted lists, remove nth node from end.
- Floyd's tortoise-and-hare algorithm detects a cycle using slow and fast pointers in O(n) time and O(1) extra space.

Reverse a singly linked list – pointer idea

```
prev = NULL; curr = head
while curr != NULL:
next = curr.next
curr.next = prev
prev = curr
curr = next
new_head = prev
```

8.4 Stack and queue

Structure	Rule	Common uses
Stack	LIFO – last in, first out	Function calls, undo, expression parsing, DFS
Queue	FIFO – first in, first out	Scheduling, BFS, buffering
Deque	Insert/remove at both ends	Sliding-window maximum, flexible queue/stack
Priority queue	Highest/lowest priority served first	Dijkstra, scheduling, top-k problems

- Balanced-parentheses checking: push opening brackets; on closing bracket, check and pop the matching opener; valid only if no mismatch and stack ends empty.
- A circular queue reuses array positions with modulo arithmetic: next = (index + 1) % capacity.
- A monotonic stack maintains increasing/decreasing values to solve next greater/smaller element problems in O(n).

8.5 Hashing

- A hash function maps a key to a bucket/index. Collisions occur when different keys map to the same bucket.
- Collision handling commonly uses separate chaining or open addressing (linear probing, quadratic probing, double hashing).
- Load factor = number of stored entries / number of buckets. High load factor can increase collisions and lookup cost.
- Hash-table search/insert/delete are average O(1) under suitable assumptions; worst case can be O(n).

DSA – trees, graphs, sorting and searching

8.6 Trees and binary search trees

- A tree is a hierarchical structure. A binary tree has at most two children per node. A BST orders keys: left subtree keys are smaller and right subtree keys are larger (duplicates depend on design).
- Inorder traversal: Left, Root, Right. Preorder: Root, Left, Right. Postorder: Left, Right, Root. Level order uses a queue.
- A balanced BST supports O(log n) search/insert/delete; a skewed BST can degrade to O(n).
- A heap is a complete binary tree with heap order. In a max-heap each parent is at least as large as its children; in a min-heap it is at most as large.
- Heap insert and extract are O(log n); peek at root is O(1).

8.7 Graphs

- A graph consists of vertices and edges; it can be directed/undirected and weighted/unweighted.
- Adjacency matrix uses $O(V^2)$ space and offers O(1) edge lookup; adjacency list uses O(V+E) space and is efficient for sparse graphs.
- BFS uses a queue and finds shortest paths by edge count in unweighted graphs. DFS uses recursion or a stack.
- Dijkstra finds shortest paths with non-negative edge weights; it is not valid for negative edge weights.
- Topological sorting applies to directed acyclic graphs (DAGs), commonly using indegree/Kahn's algorithm or DFS.

8.8 Sorting and searching

Algorithm	Average / worst time	Stable? / notes
Bubble sort	$O(n^2)$ / $O(n^2)$	Stable with standard adjacent-swap implementation; useful mainly for teaching.
Selection sort	$O(n^2)$ / $O(n^2)$	Usually unstable; O(n) swaps.
Insertion sort	$O(n^2)$ / $O(n^2)$	Stable; efficient for small or nearly sorted arrays.
Merge sort	O(n log n) / O(n log n)	Stable; typically O(n) extra array space.
Quicksort	O(n log n) / $O(n^2)$	Usually in-place; pivot choice affects balance.
Heap sort	O(n log n) / O(n log n)	In-place, generally unstable.

8.9 DSA problem-solving checklist

- Clarify constraints and input format before choosing an approach.
- Start with a simple correct solution; identify the bottleneck and optimize.
- Test empty, minimal, duplicate, sorted, reverse-sorted, negative and large inputs.
- Explain correctness intuition, time complexity and auxiliary space.
- Know the patterns: two pointers, sliding window, binary search, hashing, recursion/backtracking, BFS/DFS and dynamic programming basics.

9. Final Quick-Revision Sheet

C/C++ distinctions

Compare	Key distinction
C vs C++	C is procedural; C++ supports procedural, OOP and generic programming.
malloc vs calloc	malloc leaves bytes uninitialized; calloc initializes allocated bytes to zero.
struct vs union	Struct members have separate storage; union members share storage.
Pointer vs reference	Pointer can be null/reseated; reference is an alias and cannot be reseated.
Overloading vs overriding	Overloading changes parameter list; overriding replaces a virtual base implementation.
new/delete vs malloc/free	new/delete construct/destroy typed objects; malloc/free manage raw allocated storage.
map vs unordered_map	Ordered tree O(log n) vs hash table average O(1).

OS / DBMS / Networks formulas and facts

Area	Quick fact
Scheduling	Turnaround = Completion – Arrival; Waiting = Turnaround – CPU burst; Response = First start – Arrival.
Deadlock	Mutual exclusion + hold and wait + no preemption + circular wait.
ACID	Atomicity, Consistency, Isolation, Durability.
Normalization	1NF atomic values; 2NF no partial dependency; 3NF removes relevant transitive dependency; BCNF every determinant is a superkey.
TCP setup	SYN → SYN-ACK → ACK.
Ports	HTTP 80, HTTPS 443, DNS 53, SSH 22, DHCPv4 UDP 67/68.

DSA complexity snapshot

Operation / algorithm	Typical complexity
Array index access	O(1)
Linear search	O(n)
Binary search	O(log n)
Merge sort / heap sort	O(n log n)
Quicksort	Average O(n log n), worst $O(n^2)$
Hash lookup	Average O(1), worst O(n)
BFS / DFS	O(V + E) with adjacency lists
BST search	O(log n) balanced; O(n) skewed

Final preparation advice

- Do not memorize definitions alone. Explain each concept with a small example.
- Practise C/C++ code tracing and write programs without relying on an IDE's autocomplete.
- For every DSA problem, state the brute-force method and then the optimized method.
- Revise mistakes from the MCQ bank; repeated error patterns reveal weak topics.
- Be ready to discuss one project clearly: problem, architecture, your contribution, challenge, testing and improvement.

Accuracy note: These are general technical revision notes aligned to the shared role description and relevant software-engineering fundamentals. They are not an official Sasken syllabus or a guarantee of the exact 2027 assessment. Verify

any company-specific test instructions with the placement team.